Bases de Datos a Gran Escala

Sesión 4 · Bases de datos basadas en documentos

Diego Sevilla Ruiz
Departamento de Ingeniería y Tecnología de Computadores
Facultad de Informática · Universidad de Murcia

dsevilla@um.es

Introducción a las bases de datos de documentos

Bases de datos documentales

- Organizadas, en general, como un conjunto de **colecciones** que contienen **documentos**:
  - Tablas relacionales ↔ colecciones
  - Filas (*rows*) ↔ documentos
  - No restringen la estructura de cada documento y no tienen esquema
- Cada colección es un *array* donde cada documento tiene asociado un identificador
- La base de datos puede ver el contenido del documento, y utilizar su información como parte de las búsquedas y actualizaciones
- Documentos ⇒ formatos jerárquicos tipo JSON o XML

Bases de datos documentales (ii)

- No utilizan SQL y tienen su propio lenguaje de consulta
- Utilizaban tradicionalmente **Map-Reduce** para cálculos distribuidos; hoy es una técnica histórica, sustituida por el *framework* de agregación
- Algunas implementan otros lenguajes de consulta y procesado, como SQL++ (Couchbase, antes N1QL) y *Aggregation Framework* (MongoDB)
- Bases de datos de documentos: Couchbase, MongoDB, OrientDB

El documento como agregado

Un documento reúne lo que la aplicación lee y escribe junto: es el **agregado** de la sesión 3. Es a la vez la unidad de:

- **Acceso**: se recupera entero con una búsqueda por clave, sin `JOIN`
- **Localidad**: sus datos se almacenan contiguos
- **Atomicidad**: una escritura sobre un documento se aplica entera o no se aplica
- **Distribución**: al repartir la colección entre nodos (*sharding*), cada documento vive entero en uno

> Decidir qué entra en un documento es decidir las cuatro cosas a la vez

Esquema flexible no es ausencia de esquema

- *Schema-on-read* (sesión 3): el esquema existe, pero vive en el código que lee
- Dos documentos de una colección pueden tener campos distintos, y un campo **ausente** no es lo mismo que un campo a `null`
- Cuando conviene, la base de datos también puede **validar al escribir**. En MongoDB:

```
db.createCollection("users", { validator: { $jsonSchema: {
  required: ["Id", "DisplayName"],
  properties: { Reputation: { bsonType: "int", minimum: 0 } }
} } })
```

- El esquema pasa de obligatorio a **gradual**: se exige sólo donde aporta

Documental y relacional: dos modelos que convergen

	Relacional	Documental
Punto fuerte	`JOIN`, relaciones muchos a muchos, consultas no previstas	Agregados que se leen enteros, estructura variable
Punto débil	Recomponer un agregado exige varios `JOIN`	Duplicación; las uniones recaen en la aplicación
Ha incorporado	Columnas JSON, con consultas e índices (sesión 2)	`$lookup`, transacciones, validación de esquema

> La pregunta ya no es «SQL o NoSQL», sino qué forma tienen los datos y qué consultas hay que dejar optimizadas

Modelado de bases de datos de documentos

Modelado de datos en NoSQL

El modelado de datos debe ser:

- Realizado al mayor nivel de abstracción posible
- Independiente de la tecnología subyacente

Sin embargo, en NoSQL:

- Se tiene que tener en cuenta el diseño **distribuido**
- **Optimización guiada por las consultas**

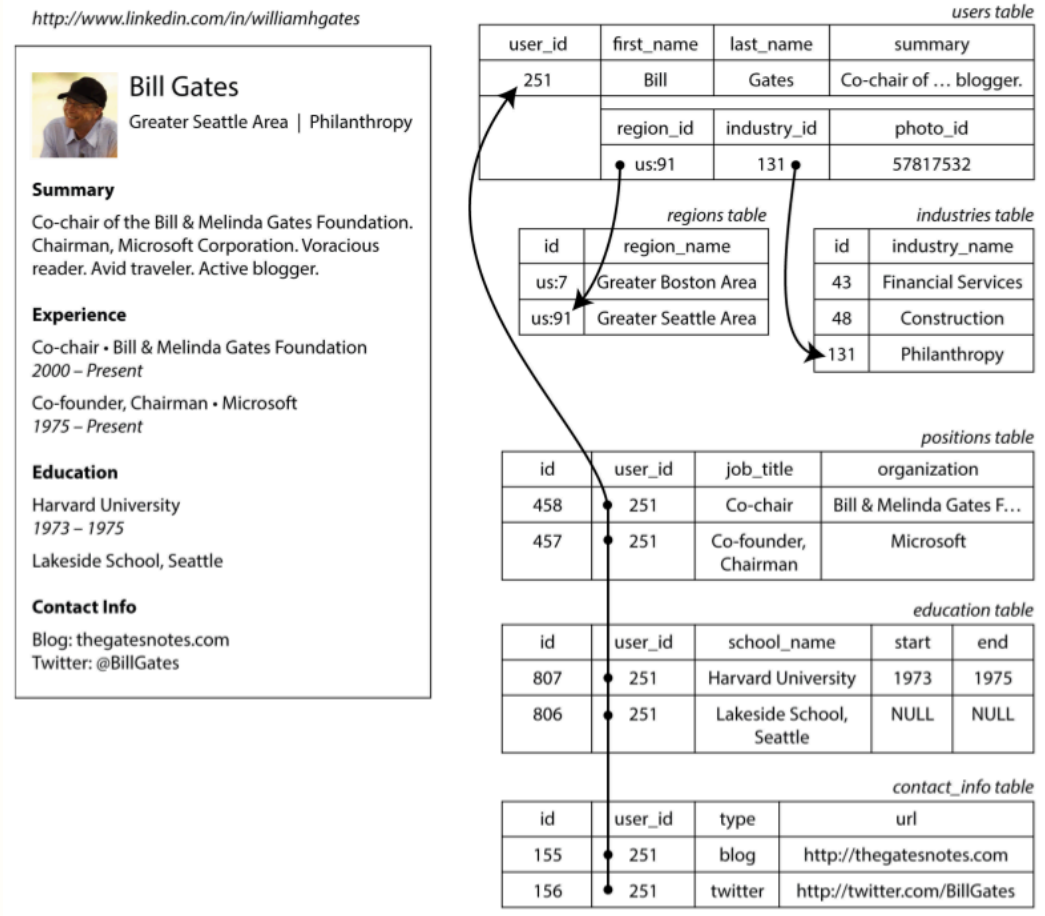
Modelado de datos en NoSQL (ii)

Con respecto al modelo de datos:

- Se mantienen los conceptos de entidad, relación, cardinalidades, etc.
- El modelado relacional se centra en especificar **qué datos tenemos y podemos ofrecer**
- El modelo NoSQL se centra en **optimizar qué consultas vamos a servir**
- Es poco costoso **duplicar (desnormalizar)** los datos si con ello se consigue **mayor eficiencia de acceso**
- **Agregar** o **referenciar** —la decisión que planteamos en la sesión 1— según qué consultas queramos tener optimizadas

Representación de un CV como tablas

Kleppmann, 2016. *Designing Data-Intensive Applications*



http://www.linkedin.com/in/williamhgates

Bill Gates

Greater Seattle Area | Philanthropy

Summary

Co-chair of the Bill & Melinda Gates Foundation. Chairman, Microsoft Corporation. Voracious reader. Avid traveler. Active blogger.

Experience

Co-chair • Bill & Melinda Gates Foundation
2000 – Present

Co-founder, Chairman • Microsoft
1975 – Present

Education

Harvard University
1973 – 1975

Lakeside School, Seattle

Contact Info

Blog: thegatesnotes.com
Twitter: @BillGates

users table

user_id	first_name	last_name	summary
251	Bill	Gates	Co-chair of … blogger.

region_id	industry_id	photo_id
us:91	131	57817532

regions table

id	region_name
us:7	Greater Boston Area
us:91	Greater Seattle Area

industries table

id	industry_name
43	Financial Services
48	Construction
131	Philanthropy

positions table

id	user_id	job_title	organization
458	251	Co-chair	Bill & Melinda Gates F…
457	251	Co-founder, Chairman	Microsoft

education table

id	user_id	school_name	start	end
807	251	Harvard University	1973	1975
806	251	Lakeside School, Seattle	NULL	NULL

contact_info table

id	user_id	type	url
155	251	blog	http://thegatesnotes.com
156	251	twitter	http://twitter.com/BillGates

Representación de relaciones uno a muchos

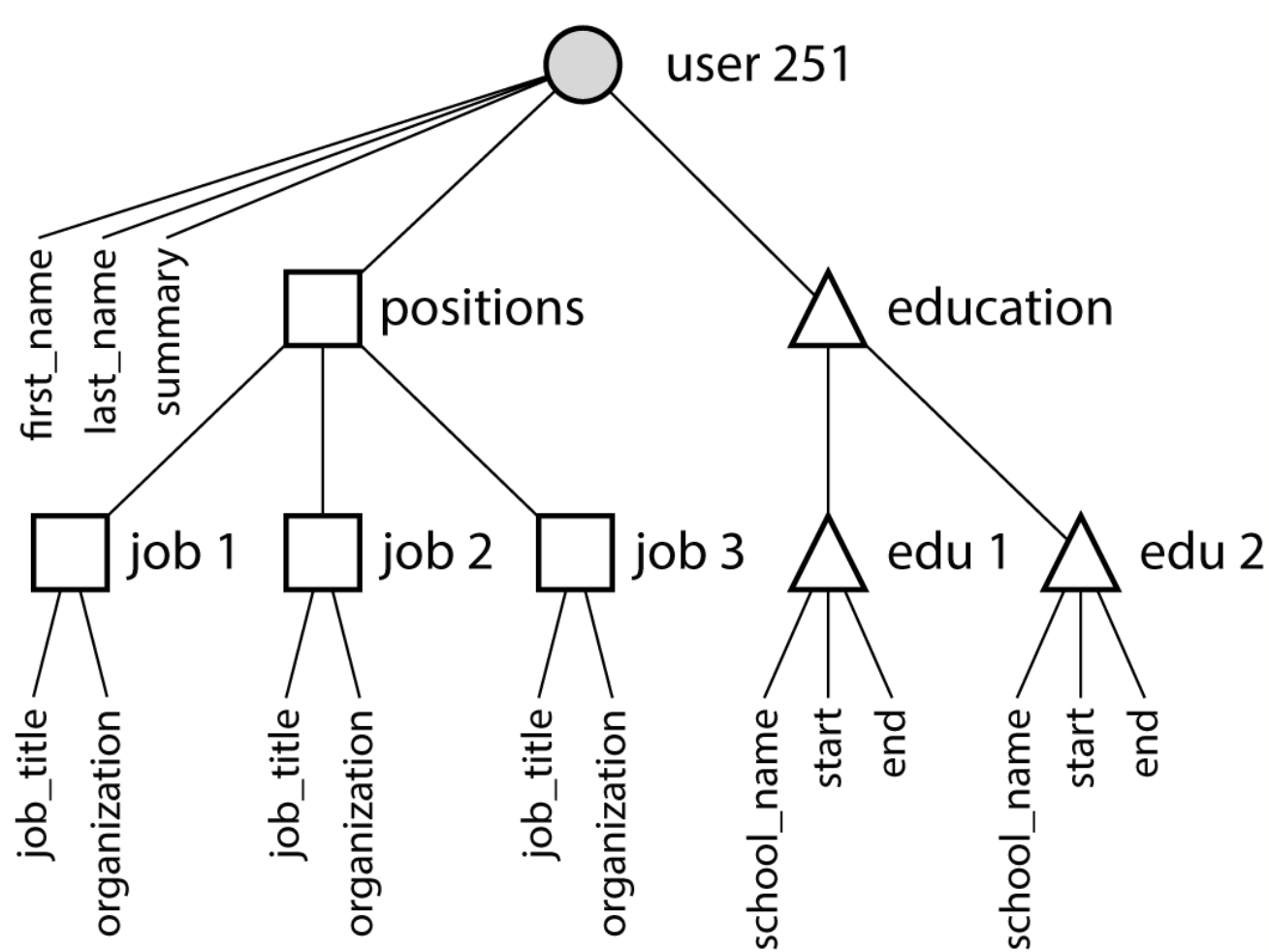
Las relaciones uno a muchos (por ejemplo, en el CV: `positions`) en el modelo relacional:

- Normalización usando varias tablas (`Positions` con `user_id`)
  - Necesidad de más de una tabla
  - Necesidad de usar `JOIN` ⇒ ineficiencia
- Los SGBDR actuales ofrecen además columnas JSON (sesión 2), con consultas e índices sobre sus campos
  - El estándar las recoge desde SQL:2016, pero cada gestor tiene su propia sintaxis
  - El documento es ahí un añadido a la fila, no la unidad del modelo

CV como un documento

```
{
  "user_id": 251,
  "first_name": "Bill",
  "last_name": "Gates",
  "summary": "Co-chair of the Bill & Melinda Gates... Active blogger.",
  "region_id": "us:91",
  "industry_id": 131,
  "photo_url": "/p/7/000/253/05b/308dd6e.jpg",
  "positions": [
    { "job_title": "Co-chair", "organization": "Bill & Melinda Gates Foundation" },
    { "job_title": "Co-founder, Chairman", "organization": "Microsoft" }],
  "education": [
    { "school_name": "Harvard University", "start": 1973, "end": 1975 },
    { "school_name": "Lakeside School, Seattle", "start": null, "end": null }],
  "contact_info": {
    "blog": "http://thegatesnotes.com",
    "twitter": "http://twitter.com/BillGates"
  }
}
```

CV como un árbol (equivalente)



Representación de relaciones: modelo de documentos

- **Modelo de documentos** → analogía del **array/mapa gigante**
- **Conjunto de documentos** (objetos complejos):
  - Un **identificador único**, campo *id*
  - Búsqueda aleatoria eficiente por clave (**referencia**)
  - Estructura jerárquica de subdocumentos contenidos → **agregación**

> Más flexibilidad que el modelo relacional: elección entre *referencia* y *agregación*

Uno a muchos (ii) — NoSQL

Relaciones **uno a muchos** (`positions`):

- **Opción 1**: agregando la tabla `positions`
- **Opción 2**: convertir las empresas en entidades y utilizar una ***referencia***

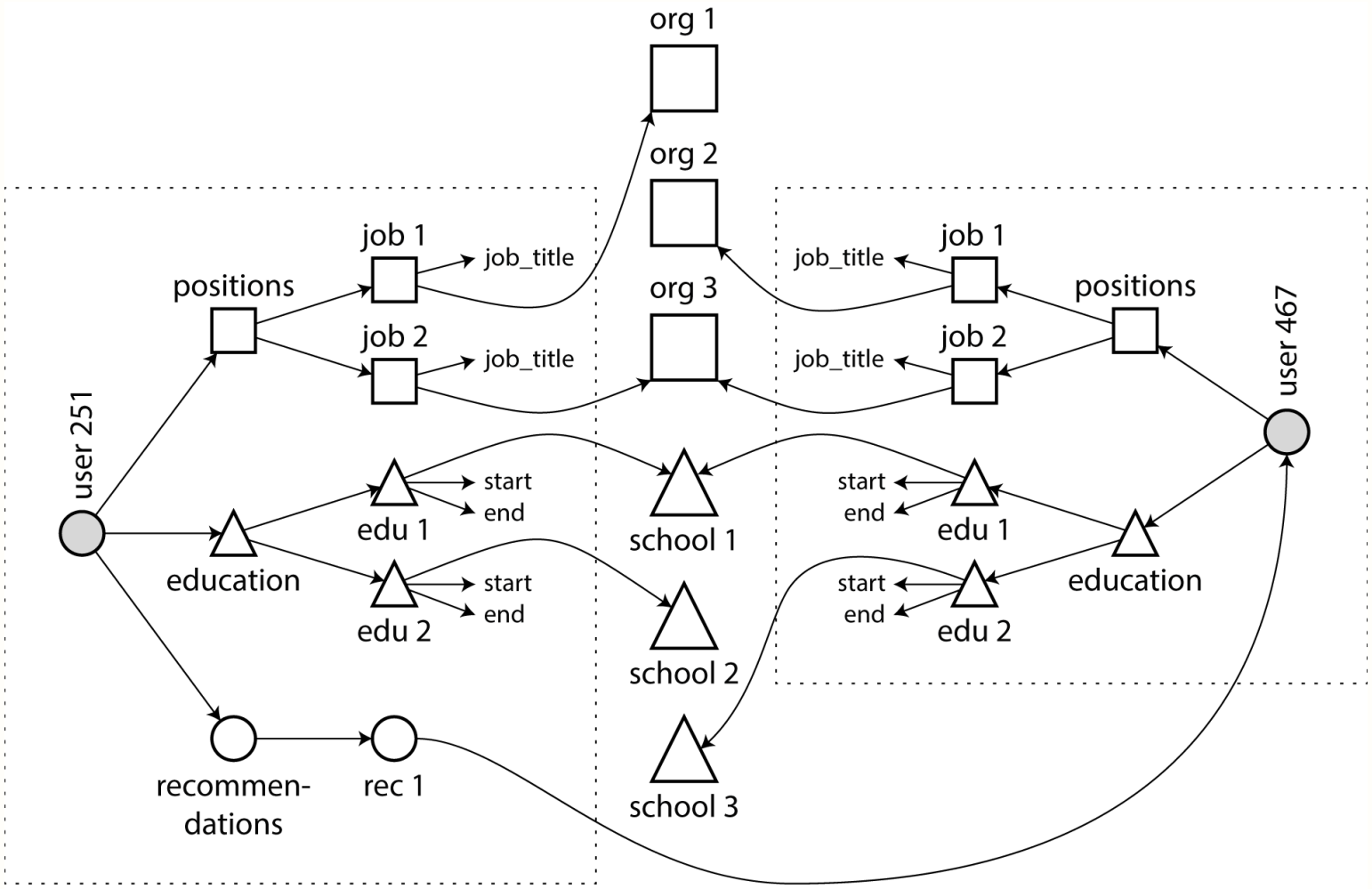
¡Modelado guiado por el acceso a datos!

- Si los elementos «muchos» tienen una estructura sencilla ⇒ **Opción 1**
- Si los elementos «muchos» se recuperan usualmente **junto con** el elemento «uno» ⇒ **Opción 1**
- Si los elementos «muchos» son relativamente grandes, o se recuperan siempre por separado ⇒ **Opción 2**

Muchos a uno y muchos a muchos

- Ejemplos: personas que viven en una región; preguntas que refieren a *tags*
- El modelo de documentos no aporta ventajas frente al modelo relacional
- Al haber muchas entidades que refieren a otra entidad, la *agregación* daría lugar a mucha **duplicación** (y a problemas de sincronización)
- ⇒ **Referencias** (sobre el `id`), similar a una FK en el modelo relacional
  - **Sin embargo**, al no haber **`JOIN`s**, la aplicación tiene que hacer más de una petición a la BD

Muchos a muchos — referencia



Introducción a MongoDB

Introducción a MongoDB

- Base de datos documental
- Basada en una representación binaria de JSON, llamada BSON
- Cada base de datos se divide en un conjunto de *colecciones*
- Una colección es un conjunto arbitrariamente grande de documentos
- Cada documento contiene un campo especial `_id`, que guarda un objeto de tipo especial `ObjectID` de BSON
- Permite la definición de bases de datos, colecciones (tablas *hash* de documentos) y subcolecciones
- Permite búsquedas secuenciales, programas MapReduce y un API propietario de consulta (llamado *aggregate*)
- Ofrece un *shell* que acepta JavaScript, y clientes en muchos lenguajes de programación

BSON: JSON con tipos

JSON sólo distingue cadenas, números, booleanos, `null`, *arrays* y objetos. BSON, el formato binario en que MongoDB guarda y transmite los documentos, añade tipos:

Tipo BSON	Para qué	En Python (`pymongo`)
`ObjectId`	Identificador único de 12 bytes: el `_id` por omisión	`bson.ObjectId`
`Date`	Instante en milisegundos (UTC)	`datetime.datetime`
`Int32`, `Int64`, `Double`	Números con tipo definido	`int`, `float`
`Decimal128`	Decimales exactos (importes)	`bson.Decimal128`
`BinData`	Datos binarios	`bytes`

Un documento ocupa como máximo **16 MB**: un límite que condiciona cuánto se puede agregar

¿Dónde está el SQL de MongoDB?

- A una base de datos relacional se le envía un **texto** en un lenguaje propio y declarativo: SQL
- MongoDB no tiene un lenguaje de consulta textual: su *shell* (`mongosh`) es un intérprete de **JavaScript**
- Lo que en SQL es sintaxis, aquí son **objetos y métodos** del lenguaje:
  - `db` es un objeto: la base de datos en uso
  - Cada colección es una **propiedad** de `db`: `db.posts` o `db["posts"]`, como la entrada de un diccionario
  - Cada documento es un **objeto** JavaScript: un diccionario de claves y valores
  - Cada operación es un **método** de la colección: `db.posts.find(...)`

Una consulta es un documento

```
SELECT Title, Score FROM Posts
WHERE PostTypeId = 1 AND Score >= 40
ORDER BY Score DESC LIMIT 10;
```

```
db.posts.find({ PostTypeId: 1, Score: { $gte: 40 } },   // WHERE
              { Title: 1, Score: 1, _id: 0 })            // SELECT
        .sort({ Score: -1 })                             // ORDER BY
        .limit(10)                                       // LIMIT
```

- El filtro, la proyección y el orden son **documentos**: datos, no texto
- Los operadores (`$gte`, `$in`, `$or`…) son claves que empiezan por `$`
- Una consulta se construye como cualquier otra estructura de datos, sin concatenar cadenas

De SQL a JavaScript: equivalencias

SQL	`mongosh` (JavaScript)
`CREATE TABLE Posts (...)`	Nada: la colección se crea al insertar
`INSERT INTO Posts VALUES (...)`	`db.posts.insertOne({...})`, `db.posts.insertMany([...])`
`SELECT * FROM Posts WHERE Id = 4`	`db.posts.find({ Id: 4 })`
`SELECT Title FROM Posts`	`db.posts.find({}, { Title: 1 })`
`WHERE Score >= 40 AND PostTypeId = 1`	`{ Score: { $gte: 40 }, PostTypeId: 1 }`
`ORDER BY Score DESC LIMIT 10`	`.sort({ Score: -1 }).limit(10)`
`UPDATE Posts SET Score = 0 WHERE Id = 4`	`db.posts.updateOne({ Id: 4 }, { $set: { Score: 0 } })`
`DELETE FROM Posts WHERE Score < 0`	`db.posts.deleteMany({ Score: { $lt: 0 } })`
`SELECT COUNT(*) FROM Posts WHERE ...`	`db.posts.countDocuments({...})`
`GROUP BY`, `HAVING`, `JOIN`	`db.posts.aggregate([...])` con `$group`, `$match`, `$lookup`
`CREATE INDEX ... ON Posts (Score)`	`db.posts.createIndex({ Score: 1 })`
`EXPLAIN SELECT ...`	`db.posts.find(...).explain()`

Qué se ejecuta y dónde

- `mongosh` evalúa el JavaScript **en el cliente** y convierte cada método en una orden para el servidor, que es otro documento:

```
{ find: "posts", filter: { Score: { $gte: 40 } }, limit: 10 }
```

- El servidor (`mongod`) no interpreta JavaScript: recibe el documento, **planifica** la consulta y elige los índices, como el optimizador de SQL
- `find` es declarativo, como un `SELECT`; `aggregate` describe una **secuencia de etapas**, más cercana a un plan escrito a mano
- Sólo `$where`, `$function` y el antiguo `mapReduce` ejecutan JavaScript en el servidor: están desaconsejados

> Cualquier lenguaje puede ser cliente: basta con saber construir esos documentos

Uso básico de MongoDB

El *shell* acepta código JavaScript. Existen una serie de objetos predefinidos:

```
show databases;
use <database>;
db.<collection>; // Permite acceder a la colección
```

Sólo nombrando las colecciones se puede acceder a ellas para consulta y actualización. Inserción de documentos:

```
db.colección.insertOne({< Objeto JSON >});
```

Actualización y consulta

Se pueden obtener los documentos de la base de datos con:

```
var objeto = db.colección.findOne({ atributos : valores})
// ó encontrar todos los que cumplen la condición:
var objs = db.colección.find({ atributos : valores})
```

Y sustituirlos por otro:

```
db.colección.replaceOne(
     { atributos: valores},// búsqueda
     objeto);                  // nuevo objeto
```

La selección del objeto se suele hacer por su `_id`, que es único

Actualización: modificadores

Para cambiar sólo parte del documento, `updateOne` (o `updateMany`) con un **modificador**:

```
db.analytics.updateOne({"url" : "www.example.com"},
                       {"$inc" : {"pageviews" : 1}})
```

También `$set`, `$unset`, `$push` (para *arrays*), `$pull` (para eliminar elementos de un *array* que cumplen un criterio)

Consulta

Las consultas se hacen con `find`:

```
db.users.find({<criterio>},
              {"username" : 1, "_id" : 0})
// =>
{
"username" : "joe",
}
```

Y se pueden utilizar condicionantes para la búsqueda:

```
db.users.find({"age": {"$gte": 18, "$lte": 30}});
db.users.find({"$or": [ {"age": {"$gte": 18} },
                        {"permission": true} ] });
```

De JavaScript a `pymongo`

Los *drivers* ofrecen las mismas operaciones que el *shell*, con las convenciones de cada lenguaje. En Python, los nombres pasan de *camelCase* a *snake_case*:

`mongosh` (JavaScript)	`pymongo` (Python)
`insertOne`, `insertMany`	`insert_one`, `insert_many`
`findOne`	`find_one`
`updateOne`, `updateMany`, `replaceOne`	`update_one`, `update_many`, `replace_one`
`deleteOne`, `deleteMany`	`delete_one`, `delete_many`
`countDocuments`	`count_documents`
`createIndex`, `dropIndex`	`create_index`, `drop_index`
`find`, `aggregate`, `distinct`, `watch`	Igual: son una sola palabra

Con cualquier ejemplo en JavaScript de la documentación o de un libro, basta aplicar la regla

De JavaScript a `pymongo`: qué cambia en los documentos

- Un documento es un `dict`. Los operadores y los nombres de los campos **no cambian**, pero las claves se escriben entre comillas: `{ $gte: 40 }` → `{"$gte": 40}`
- `true`, `false`, `null` → `True`, `False`, `None`
- `ISODate("2026-10-08")` → `datetime(2026, 10, 8)`
- Las opciones dejan de ser un documento y pasan a ser argumentos con nombre
- Con el cliente async, cada operación que va al servidor lleva `await`

```
db.posts.updateOne({ Id: 4 }, { $inc: { Score: 1 } }, { upsert: true })
```

```
await db.posts.update_one({"Id": 4}, {"$inc": {"Score": 1}}, upsert=True)
```

Lo mismo, con `pymongo` (async): un adelanto de la práctica

Esto es literalmente lo primero que ejecutaremos en la sesión de prácticas, sobre una colección `personas` de prueba. La conexión se abre con `AsyncMongoClient` y cada operación lleva `await`:

```
personas: AsyncCollection = db.personas
await personas.insert_one({"nombre": "Juan", "edad": 25})
await personas.insert_one({"nombre": "Diego", "apellido": "Sevilla", "edad": 30})
await personas.find_one({"nombre": "Diego"})
```

```
await personas.update_one({"nombre": "Diego"}, {"$set": {"edad": 35}})
await personas.update_many(
    {"edad": {"$gte": 18}}, {"$set": {"mayor_de_edad": True}}
)
```

Consultas MapReduce (histórico)

MongoDB lo ofrecía antes de tener un *framework* de agregación propio; hoy está desaprobado a favor de `aggregate()` (más adelante en esta sesión), pero conviene reconocerlo si aparece en código antiguo

```
db.colección.mapReduce(<función map>, <función reduce>, <opciones>)
```

```
var thm_map = function () {
    var date = new ISODate(this.CreationDate);
    var dayOfWeek = date.getDay();
    var hour = date.getHours();

    emit( dayOfWeek + '-' + hour, 1);
};

var thm_reduce = function (key, values) {
    return Array.sum(values);
}
```

Uso de MongoDB desde `pymongo`

En las prácticas usamos siempre el cliente asyn (`AsyncMongoClient`):
toda operación sobre una colección lleva `await` delante

Métodos de búsqueda

El método de búsqueda principal es `find()`, que tiene muchas opciones. En general permite especificar:

- El filtro de búsqueda
- Ordenación de resultados por algún campo
- Proyección, para no obtener todos los campos del documento
- Número de resultados máximo (*limit*)
- Número de elementos iniciales a ignorar (*skip*)
- El tamaño del *batch*

Como la variabilidad es muy grande, se ven ejemplos en la hoja Jupyter de prácticas y en la documentación

Métodos de búsqueda: calificadores

`find()` admite calificadores complejos que unen «objetos», no valores:

- `$and`, `$or`, `$not`

Y otros que se refieren a valores:

- `$lt` (menor), `$lte` (menor o igual), `$gt` (mayor), `$gte` (mayor o igual), `$regex` (expresión regular)

```
await posts.find_one({"body": {"$regex" : "[Mm]ongo"}})
```

Condición compuesta (limitando la salida a 10 documentos):

```
await posts.find({"$and": [{"PostTypeId": 2},
                           {"Id" : {"$gte" : 100}} ]}).limit(10).to_list()
```

Métodos de búsqueda: ejemplos introductorios

Encontrar el primer documento en la colección `customers`:

```
client = AsyncMongoClient("mongodb://localhost:27017/")
mydb = client["mydatabase"]
mycol = mydb["customers"]
x = await mycol.find_one()
```

Encontrar todos los documentos en la colección `customers`:

```
async for x in mycol.find():
  print(x)
```

Métodos de búsqueda: proyección

También podemos indicar qué campos queremos obtener de una determinada búsqueda:

```
async for x in mycol.find({},{ "_id": 0, "name": 1, "address": 1 }):
  print(x)
```

Ejemplos básicos

Dada la siguiente colección:

```
{ "_id": "manzanas", "cantidad": 5 }
{ "_id": "plátanos", "cantidad": 7 }
{ "_id": "naranjas", "cantidad": 8}
{ "_id": "aguacate", "cantidad": 14}
```

Documentos con `cantidad` superior a 4:

```
db.collection.find( { "cantidad": { "$gt": 4 } } )
```

Filtrado por rango de fechas

Suponiendo una colección `bios` cuyos documentos tienen esta forma:

```
{
    "_id": <valor>,
    "nombre": { "nombre_propio": <string>, "primer_apellido": <string>},
    "nacimiento": <ISODate>,
    "muerte": <ISODate>,
    "profesion": <valor>,
    "trabajos": [<string>,...]
}
```

```
from datetime import datetime
db.bios.find( { "nacimiento": { "$gt": datetime(1940,1,1),
 "$lt": datetime(1960,1,1) } } )
```

El operador `$in`

Permite buscar documentos con atributos dentro de una lista de valores:

```
db.bios.find(
    { "profesion": { "$in": [ 'jedi', 'samurai' ] } }
)
```

Campos anidados y *arrays*

También podemos realizar búsquedas en campos anidados:

```
db.bios.find( { "nombre.primer_apellido": "Skywalker" } )
db.bios.find(
    { "nombre": { "nombre_propio": "Akira", "primer_apellido": "Kurosawa" } }
)
```

Las búsquedas pueden realizarse también sobre campos cuyos valores son *arrays*:

```
db.bios.find({"trabajos": "star_wars"})
db.bios.find({"trabajos": { "$in": [ "star wars", "alien" ]} })
db.bios.find({"trabajos": { "$all": [ "star wars", "the clone wars" ]}})
db.bios.find({"trabajos": { "$size": 4 }})  # 'trabajos' debe tener 4 elementos
```

`$not`, `$and`, `$or`

`$not` es útil para buscar elementos que NO cumplan una condición:

```
# Documentos cuyo precio no sea superior a 1,99€ o que no tengan campo "precio"
db.inventory.find( { "precio": { "$not": { "$gt": 1.99 } } } )
```

`$and`, `$or` y `$not` pueden combinarse para condiciones complejas:

```
db.inventory.find( {
    "$and": [
        {"$or":[{"cantidad":{"$lt":10}}, {"cantidad":{"$gt":50} }]},
        {"$or":[{"vendido": True}, {"precio": {"$lt": 5}}]}
    ]
} )
```

¿Qué documentos devolverá la consulta de arriba?

Métodos de inserción y actualización

`pymongo` ofrece métodos para inserción y actualización:

- `insert_one()`, `insert_many()` (*batch*)
- `update_one()`: actualiza un objeto con nuevos campos. Se crea si `upsert=True`
- `update_many()`: aplica nuevos valores calculados a un conjunto de objetos

Inserción: ejemplos básicos

Al insertar un elemento podemos obtener su `_id`:

```
micoleccion = db["clientes"]
midict = { "nombre": "Luke", "direccion": "Tatooine, 83" }
elem = await micoleccion.insert_one(midict)
print(elem.inserted_id)
```

Para insertar un conjunto de documentos a la vez, se incluyen en un *array*:

```
milista = [ { "nombre": "Amy", "direccion": "Apple st 652"},
            { "nombre": "Hannah", "direccion": "Mountain 21"},
            { "nombre": "Michael", "direccion": "Valley 345"} ]
elems = await micoleccion.insert_many(milista)
print(elems.inserted_ids)
```

Actualización: ejemplos básicos

Debemos especificar un criterio de búsqueda y los campos y valores a actualizar:

```
micoleccion = db["clientes"]
mifiltro = { "direccion": "Valley 345" }
nuevosvalores = { "$set": { "direccion": "Rue del percebe, 13" } }
await micoleccion.update_one(mifiltro, nuevosvalores)
```

Para actualizar múltiples documentos, la estructura es la misma:

```
mifiltro = { "direccion": { "$regex": "^S" } }
nuevosvalores = { "$set": { "nombre": "Rey" } }
x = await micoleccion.update_many(mifiltro, nuevosvalores)
print(x.modified_count, "documentos actualizados.")
```

Índices, `explain()`

- La llamada `.explain()` sobre cualquier búsqueda muestra el plan de ejecución
- Se puede crear un índice si la búsqueda por ese campo va a ser crítica
- Se pueden crear más índices:
  - `ASCENDING`
  - `DESCENDING`
  - `HASHED`
  - y geoespaciales

`explain()`: leer `totalDocsExamined`

Antes de crear el índice, cada búsqueda por `Reputation` recorre toda la colección:

```
plan: dict = await users.find({"Reputation": 100}).explain()
plan["executionStats"]["totalDocsExamined"]  # tantos como usuarios haya
```

Después de crear el índice, la misma búsqueda examina sólo lo necesario:

```
await users.create_index([("Reputation", 1)])
plan = await users.find({"Reputation": 100}).explain()
plan["executionStats"]["totalDocsExamined"]  # sólo los que tienen Reputation = 100
```

En la práctica compararemos además un índice **compuesto** `(Reputation, Views)`

Colecciones tipadas con `TypedDict`

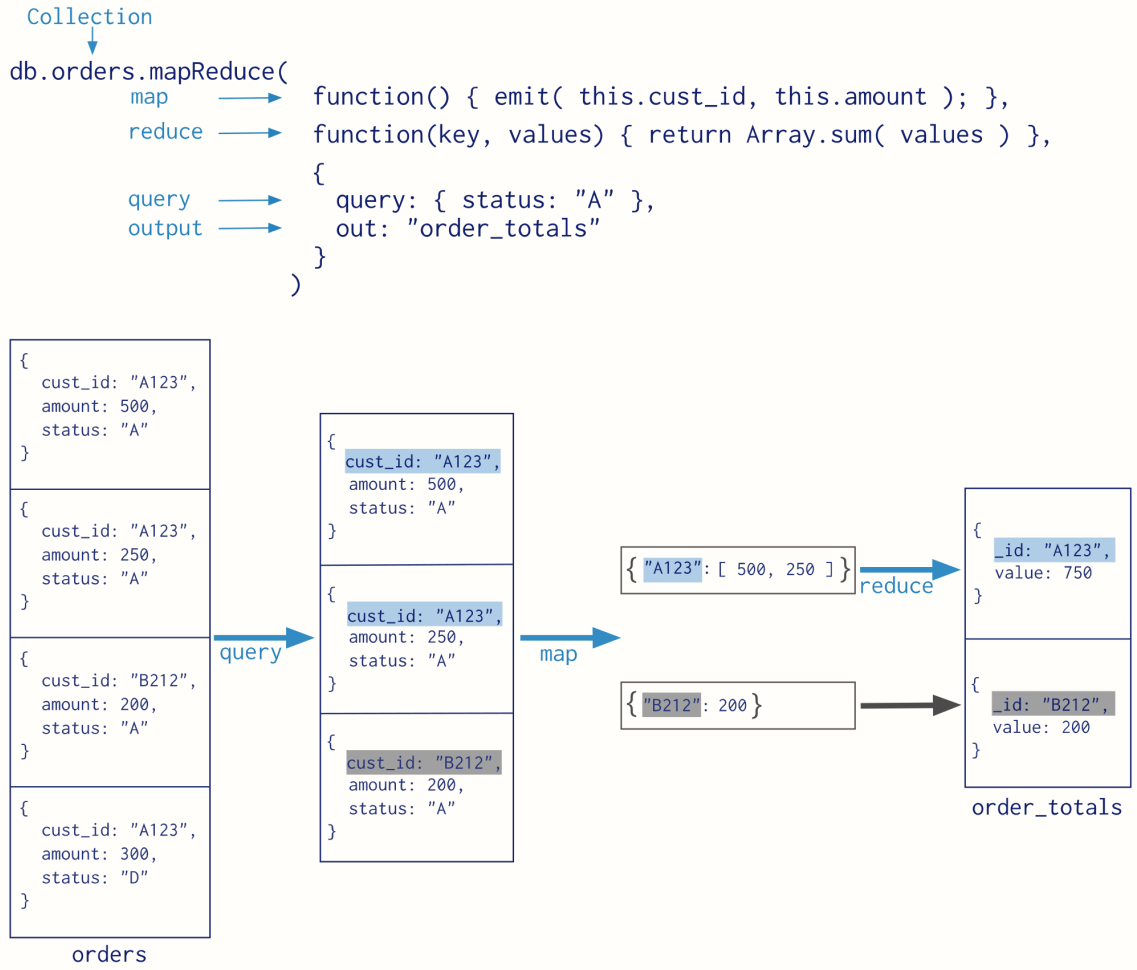
`pymongo` deja anotar una colección con el tipo de sus documentos, con la ventaja habitual del tipado: autocompletado y comprobación estática, sin cambiar ninguna consulta:

```python
class User(TypedDict):
    Id: int
    DisplayName: str
    Reputation: int
    WebsiteUrl: NotRequired[str]  # puede faltar: no todos los usuarios lo rellenan

users_t: AsyncCollection[User] = db["users"]
user: User | None = await users_t.find_one({"Id": 22656})
```

`NotRequired` marca los campos que pueden **faltar** del documento, que en MongoDB no es lo mismo que valer `None`: es la misma distinción de la sesión 1 entre ausencia y `NULL`

Map-Reduce


Collection
db.orders.mapReduce(
map
function() { emit(this.cust_id, this.amount); },
reduce
function(key, values) { return Array.sum(values) },
{
query
query: { status: "A" },
output
out: "order_totals"
}
)
{ cust_id: "A123", amount: 500, status: "A" }
{ cust_id: "A123", amount: 250, status: "A" }
{ cust_id: "B212", amount: 200, status: "A" }
{ cust_id: "A123", amount: 300, status: "D" }
orders
query
{ cust_id: "A123", amount: 500, status: "A" }
{ cust_id: "A123", amount: 250, status: "A" }
{ cust_id: "B212", amount: 200, status: "A" }
map
{ "A123": [500, 250] }
{ "B212": 200 }
reduce
{ _id: "A123", value: 750 }
{ _id: "B212", value: 200 }
order_totals

Map-Reduce en `pymongo` (histórico)

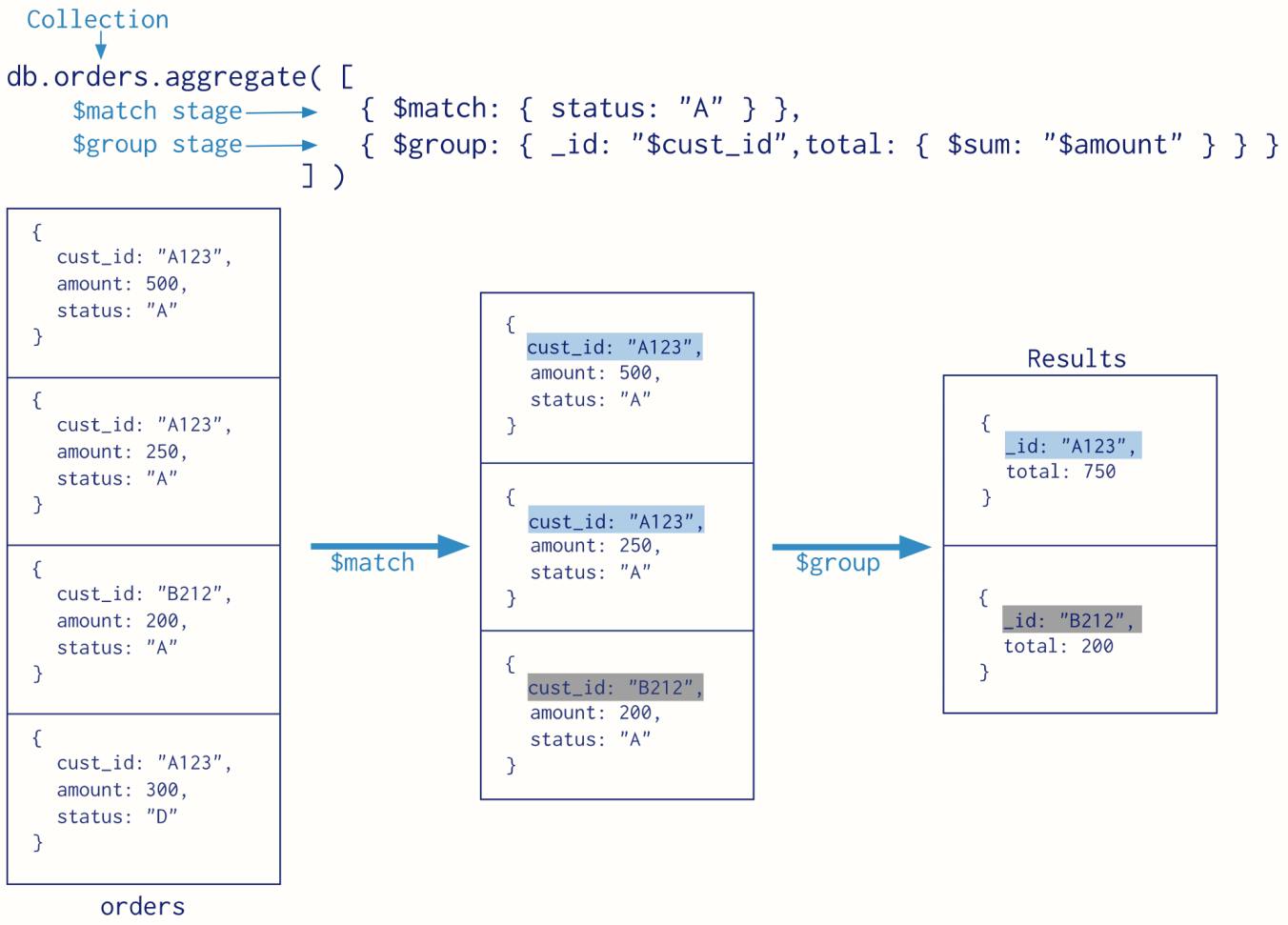
- La función `map_reduce()` de cada colección permitía ejecutar procesamientos que guardan los resultados en otra colección o los devuelven en vivo; `pymongo` 4 la eliminó
- Como el código se ejecuta dentro del motor de la base de datos, hay que enviarlo en JavaScript
- Ejemplo: histograma del tamaño del texto de los *posts*
- **No se usa en las prácticas**: el mismo cálculo se hace con el *framework* de agregación, que veremos a continuación y es lo que usaremos siempre

Map-Reduce en `pymongo` (ii)

```
from bson.code import Code
map = Code(
    '''function () {
            if ('Body' in this)
                emit(this.Body.length, 1)
            else
                emit(0,1)
        }''')
reduce = Code(
    '''function (key, values) {
             return Array.sum(values);
         }''')
results = posts.map_reduce(map, reduce, "myresults")
```

Ejercicio: ¿y la media cómo sería?

Framework de agregación



Framework de agregación

- Etapas (*stages*) del *pipeline*: documentación de agregación
- Operadores dentro de las etapas: documentación de operadores

```
await posts.aggregate( [
 {'$project' : { 'Id' : 1 }},
 {'$limit': 20}
])
```

Las operaciones de agregación pueden convertirse en complejas, con muchos operadores donde elegir: algunos poco frecuentes como `$ifNull` y `$strLenBytes`, y otros más complejos como `$expr` y `$cond`

Framework de agregación: ejemplo

```
by_length = await posts.aggregate( [
  {'$project': {
    'Body' : {'$ifNull' : ['$Body', '']}
  }},
  {'$project' : {
    'id' : {'$strLenBytes': '$Body'},
    'value' : {'$literal' : 1}
  }},
  {'$group' : {
    '_id' : '$id',
    'count' : {'$sum' : '$value'}
  }},
  {'$sort' : { '_id' : 1}}
  ])
```

Filtrar y unir colecciones

Se pueden añadir etapas; por ejemplo, **filtrar** *posts*:

```
{'$match': { 'Body' : {'$regex': 'HBase'}}}
```

`$lookup` permite el acceso a otra (o a la misma) colección: equivalente a un `JOIN`. Listar los *posts* con `Score` mayor o igual a 40 junto con el usuario que lo ha hecho:

```
await posts.aggregate( [
  {'$match': { 'Score' : {'$gte': 40}}},
  {'$lookup': {
    'from': "users",
    'localField': "OwnerUserId",
    'foreignField': "Id",
    'as': "owner"}
  }])
```

`$lookup`: resultado y `$unwind`

El resultado devuelve el campo `owner` relleno con cada usuario, en un *array*:

```
{
  'Id' : xxx, 'Title': ...,
  'owner': [ { 'Id': yyy, 'DisplayName': "zzz" ... }, ... ], ...
}
```

Esto es así porque MongoDB no sabe si el `foreignField` generará más de un elemento. La etapa `$unwind` convierte el *array* en varios documentos, uno por elemento:

```
... (resto del pipeline)
{ "$unwind" : "$owner" },
...
```

`$unwind`: resultado

```
{
  'Id' : xxx, 'Title': ...,
  'owner': { 'Id': yyy, 'DisplayName': "zzz" ... }, ...
}
```

Ha desaparecido el *array*: se genera un documento igual para cada valor que `owner` tuviera

Framework de agregación — ejemplo (i)

Ordenar por fecha ascendente los *posts* con el *tag* «java», de más antiguo a más reciente:

```
posts = db["posts"]
pipeline = [
    { "$match": { "Tags": {"$regex": "<java>"} } },
    { "$sort": { "CreationDate": pymongo.ASCENDING } }
]
results = await posts.aggregate(pipeline)
```

Si ahora quisiéramos limitar la salida a un único documento:

```
pipeline = [
    { "$match": { "Tags": {"$regex": "<java>"} } },
    { "$sort": { "CreationDate": pymongo.ASCENDING } },
    { "$limit": 1 }
]
```

Framework de agregación — ejemplo (ii)

Se puede usar también `$group` para ver los *posts* de java de cada año:

```
stage_match_tag = {
      "$match": { "Tags": {"$regex": "<java>"} }
   }
stage_group_year = {
   "$group": {
         "_id": {"$year" : "$CreationDate"},
         "post_count": { "$sum": 1 }}}
stage_sort_year_ascending = {
  "$sort": {"_id": pymongo.ASCENDING}
}
pipeline = [stage_match_tag, stage_group_year, stage_sort_year_ascending]
results = await posts.aggregate(pipeline)
```

Consistencia, transacciones y replicación

De ACID a BASE, y dónde se sitúa MongoDB

ACID frente a BASE

	ACID	BASE
Siglas	Atomicidad, Consistencia, Aislamiento (*Isolation*), Durabilidad	*Basically Available, Soft state, Eventual consistency*
Prioridad	**Corrección**: nadie ve un estado intermedio	**Disponibilidad**: responder siempre, aunque no sea con el último dato
Tras una escritura	Toda lectura posterior ve el valor nuevo	Las réplicas convergen al cabo de un tiempo
Quién asume el coste	El sistema: coordinación entre nodos, que limita la escala	La aplicación: debe tolerar lecturas antiguas y conflictos
En el curso	MySQL con InnoDB (sesión 2)	Cassandra leyendo y escribiendo a `ONE` (sesión 5)

> No son dos bandos, sino los extremos de un eje: cada sistema, y a menudo cada operación, elige su punto

CAP: por qué hay que elegir

Con los datos replicados en varios nodos, si la red se **parte** (*Partition*), un nodo que no alcanza a los demás sólo puede:

- **C** (*Consistency*): rechazar la operación, para no devolver un dato quizá obsoleto ni aceptar una escritura que los demás no verán
- **A** (*Availability*): responder con lo que tiene, y reconciliar después

Y cada sistema toma partido:

- MongoDB se inclina por **C**: sólo el nodo primario acepta escrituras, y sin mayoría no hay primario
- Cassandra se inclina por **A**: cualquier réplica acepta escrituras (sesión 5)
- Mientras la red funciona no hay que renunciar a ninguna, pero se sigue eligiendo entre consistencia y **latencia**: esperar o no a las demás réplicas

MongoDB: el documento es la transacción

- Una escritura sobre **un** documento es siempre atómica, aunque cambie varios campos, subdocumentos y *arrays*
- Es la apuesta de la sesión 3: transacciones limitadas al objeto complejo

```
db.cuentas.updateOne(
  { _id: 1, saldo: { $gte: 100 } },      // la condición y los dos cambios
  { $inc: { saldo: -100 },               // se aplican juntos, o no se aplican
    $push: { movimientos: { importe: -100, fecha: new Date() } } })
```

- Por eso lo que debe cambiar junto se **agrega** en un mismo documento
- La transferencia de la sesión 2, entre **dos** cuentas, toca dos documentos: ya no basta

MVCC: leer sin bloquear

- *Multi-Version Concurrency Control*: una escritura no sobrescribe el dato, crea una **versión** nueva
- Cada operación lee una **instantánea** (*snapshot*): las versiones confirmadas cuando empezó
- Los lectores no bloquean a los escritores, ni los escritores a los lectores
- Es la técnica de InnoDB (MySQL) y de PostgreSQL. En MongoDB la aplica su motor de almacenamiento, **WiredTiger**
- En WiredTiger el control de concurrencia es **optimista** y por **documento**:
  - Dos escrituras a documentos distintos no se estorban
  - Dos escrituras al mismo documento: una se aplica y la otra sufre un *write conflict*, que MongoDB reintenta sin que el cliente lo note
- Sobre esa base (atomicidad por documento e instantáneas) se construyen las transacciones

Transacciones multidocumento

- Desde MongoDB 4.0: ACID sobre varios documentos y colecciones, con aislamiento de instantánea
- Cada operación lleva `session=`; sin él se ejecuta **fuera** de la transacción

```
async def transferir(session: AsyncClientSession) -> None:
    await cuentas.update_one(
        {"_id": 1}, {"$inc": {"saldo": -100}}, session=session)
    await cuentas.update_one(
        {"_id": 2}, {"$inc": {"saldo": 100}}, session=session)

async with client.start_session() as session:
    await session.with_transaction(transferir)
```

- `with_transaction` confirma si la función termina, deshace si lanza una excepción y **reintenta** ante un conflicto de escritura

Transacciones: disponibles, pero no por omisión

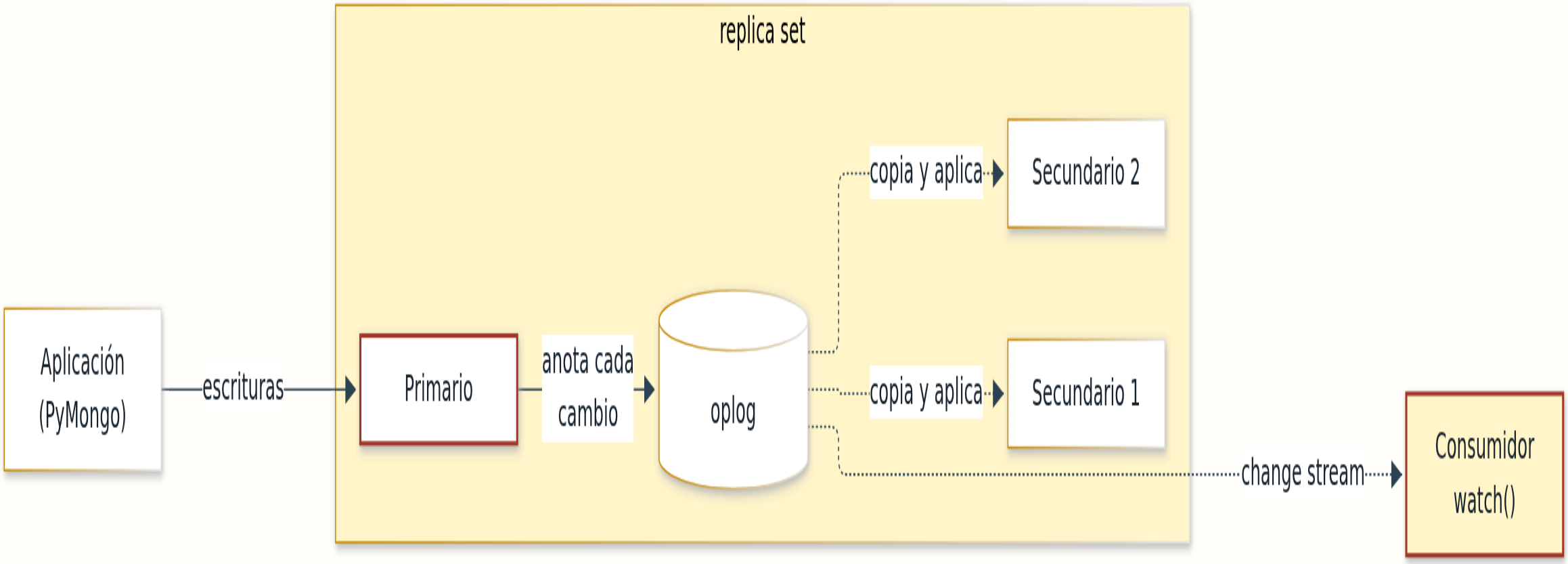
- Exigen un ***replica set*** (aunque sea de un nodo): el servidor aislado de las prácticas las rechaza
- Son más costosas que escribir un documento, y más aún si abarcan varios *shards* (*commit* en dos fases)
- Están pensadas para ser cortas: por omisión se abortan a los 60 segundos
- Un conflicto de escritura aborta la transacción entera: el código debe poder repetirse

> En el modelo relacional se normaliza y la transacción recompone la unidad. En el documental la unidad es el documento, y la transacción multidocumento es la excepción

Replicación: el *replica set*

- Un nodo **primario** recibe todas las escrituras; los **secundarios** mantienen copias
- El primario anota cada cambio en el ***oplog***: una colección de tamaño limitado con las operaciones, en orden
- Los secundarios copian el *oplog* y lo aplican: replicación **asíncrona**
- El *oplog* anota efectos, no órdenes: un `$inc` se guarda como el valor resultante, para que aplicarlo dos veces dé lo mismo (**idempotencia**)
- Si el primario cae, los secundarios **eligen** otro por mayoría: por eso se despliegan en número impar

El *oplog*: un registro, varios lectores



Los secundarios y los *change streams* leen el mismo registro de cambios

Consistencia ajustable por operación

Ajuste	Qué decide	Valores habituales
Write concern	Cuántos nodos confirman la escritura antes de responder al cliente	`w=1` (sólo el primario), `w="majority"` (por omisión desde MongoDB 5.0)
Read concern	Qué datos se pueden leer	`"local"` (lo último del nodo; por omisión), `"majority"` (confirmado por la mayoría), `"snapshot"`
Read preference	A qué nodo se pregunta	`primary` (por omisión), `secondary`, `nearest`

- Con `w=1`, una escritura confirmada puede **perderse** si el primario cae antes de replicarla
- Leer de un secundario puede devolver un dato **antiguo**: consistencia eventual, por elección

> Es el mismo mando que los niveles de consistencia de Cassandra (sesión 5)

Consistencia ajustable: en `pymongo`

Los ajustes se fijan al obtener la colección: conviven en una misma aplicación

```python
from pymongo import ReadPreference, WriteConcern
from pymongo.read_concern import ReadConcern

# Un contador de visitas: rápido, y no pasa nada si se pierde alguna
visitas: AsyncCollection = db.get_collection(
    "visitas", write_concern=WriteConcern(w=1),
    read_preference=ReadPreference.SECONDARY_PREFERRED,
)

# Un saldo: sólo lo que ha confirmado la mayoría
cuentas: AsyncCollection = db.get_collection(
    "cuentas", write_concern=WriteConcern(w="majority"),
    read_concern=ReadConcern("majority"),
)
```

Change streams: suscribirse a los cambios

- Una aplicación se **suscribe** a los cambios de una colección, de una base de datos o de todo el despliegue
- En lugar de preguntar cada cierto tiempo (*polling*), el servidor le envía cada `insert`, `update`, `replace` y `delete`
- Los cambios llegan **en orden** y sólo cuando la mayoría los ha confirmado: ya no se pueden deshacer
- `watch()` devuelve un cursor que no se agota: se queda esperando el siguiente cambio

```
async with await personas.watch() as stream:
    async for cambio in stream:
        print(cambio["operationType"], cambio["documentKey"])
```

Change streams: un ejemplo

Una tarea se queda escuchando los cambios de `personas`:

```
async def vigilar() -> None:
    async with await personas.watch() as stream:
        async for cambio in stream:
            print(cambio["operationType"], cambio.get("updateDescription"))

vigilante: Task[None] = create_task(vigilar())
```

Y cualquier escritura posterior, de este cliente o de otro, le llega:

```
await personas.update_one({"nombre": "Diego"}, {"$inc": {"edad": 1}})
```

```
update {'updatedFields': {'edad': 36}, 'removedFields': [], 'truncatedArrays': []}
```

Llega el valor resultante (`36`), no el `$inc`: es la forma idempotente del *oplog*

Change streams: filtrar y reanudar

`watch()` admite un *pipeline* de agregación para recibir sólo algunos cambios, y cada evento trae en `_id` un **token de reanudación**:

```
solo_altas: list[dict[str, Any]] = [{"$match": {"operationType": "insert"}}]
token: Mapping[str, Any] | None = await leer_ultimo_token()

async with await personas.watch(solo_altas, resume_after=token) as stream:
    async for cambio in stream:
        await procesar(cambio["fullDocument"])
        await guardar_token(cambio["_id"])
```

- Si el consumidor se cae, retoma donde se quedó, siempre que ese punto siga en el *oplog*
- Tras reanudar, un cambio puede llegar **dos veces**: el consumidor debe ser idempotente

Change streams y replicación

- Un *change stream* es la vista pública del *oplog*: por eso exige un *replica set*, conserva el orden y permite reanudar
- Su consumidor es, en la práctica, **una réplica más**, que decide qué hacer con cada cambio:
  - Mantener un índice externo: así se alimenta `mongot`, el proceso de búsqueda de texto y vectorial de MongoDB
  - Invalidar una caché, avisar a otra aplicación o mantener una colección derivada
  - Copiar los datos a otro sistema: *Change Data Capture* (CDC), que se estudia en TCDM
- Todo lo que cuelga de un *change stream* es **eventualmente consistente** respecto del primario

> En Cassandra no hay primario ni un registro único y ordenado: cada nodo tiene su *commit log*. Compararemos su captura de cambios con ésta en la sesión 5

Referencia rápida de `pymongo`

De lo más sencillo a `aggregate`, para tener a mano en las prácticas

Referencia: conexión y navegación

```python
from pymongo import AsyncMongoClient
from pymongo.asynchronous.collection import AsyncCollection
from pymongo.asynchronous.database import AsyncDatabase

client: AsyncMongoClient = AsyncMongoClient("mongo", 27017)
db: AsyncDatabase = client.stackoverflow     # o client["stackoverflow"]
posts: AsyncCollection = db.posts            # o db["posts"]

await client.list_database_names()
await db.list_collection_names()
await posts.count_documents({})              # {} es el filtro vacío: todos
await client.close()
```

- Bases de datos y colecciones se crean solas al escribir en ellas por primera vez
- Obtener `db` o `posts` no viaja al servidor: por eso no lleva `await`

Referencia: `find_one` y `find`

```
await posts.find_one({"Id": 4})                      # un documento, o None
await posts.find({"PostTypeId": 1}).to_list()        # todos, en una lista
async for post in posts.find({"PostTypeId": 1}):     # uno a uno, sin cargarlos todos
    print(post["Title"])
```

`find` no lleva `await`: devuelve un **cursor**, que no consulta hasta que se recorre. Antes se le puede añadir orden, salto y límite:

```
cursor: AsyncCursor = (
    posts.find({"PostTypeId": 1, "Score": {"$gte": 40}},    # filtro: WHERE
               {"Title": 1, "Score": 1, "_id": 0})          # proyección: SELECT
    .sort("Score", pymongo.DESCENDING)                      # ORDER BY Score DESC
    .skip(20)                                               # OFFSET 20
    .limit(10)                                              # LIMIT 10
)
await cursor.to_list()
```

Referencia: operadores de filtro

Para	Operadores	Ejemplo
Comparar	`$eq` `$ne` `$gt` `$gte` `$lt` `$lte`	`{"Score": {"$gte": 40}}`
Lista de valores	`$in` `$nin`	`{"PostTypeId": {"$in": [1, 2]}}`
Combinar condiciones	`$and` `$or` `$nor` `$not`	`{"$or": [{"Score": 0}, {"ViewCount": 0}]}`
Existencia del campo	`$exists`	`{"WebsiteUrl": {"$exists": True}}`
Texto	`$regex`	`{"Title": {"$regex": "[Mm]ongo"}}`
Arrays	`$all` `$size` `$elemMatch`	`{"trabajos": {"$all": ["star wars", "alien"]}}`
Campo anidado	Notación con punto	`{"nombre.primer_apellido": "Skywalker"}`

Son los mismos en `find`, `find_one`, `count_documents`, `update_one`, `update_many`, `delete_many` y en la etapa `$match` de `aggregate`

Referencia: `$and`, `$or` y `limit`

```
{"PostTypeId": 2, "Score": {"$gte": 10}}     # varias claves: Y implícito
{"Score": {"$gte": 10, "$lte": 20}}          # rango: dos condiciones sobre un campo
{"$or": [{"Score": {"$gte": 40}}, {"ViewCount": {"$gt": 1000}}]}
{"$and": [{"Tags": {"$regex": "java"}}, {"Tags": {"$regex": "sql"}}]}
```

- `$and` sólo hace falta para repetir una clave: en un `dict` de Python, la segunda sustituiría a la primera sin avisar
- `$and` y `$or` reciben una **lista** de filtros, y se pueden anidar

```
await posts.find(
    {"$and": [
        {"PostTypeId": 1},
        {"$or": [{"Score": {"$gte": 40}}, {"ViewCount": {"$gt": 1000}}]},
    ]}
).limit(10).to_list()      # limit corta en el servidor: sólo viajan diez documentos
```

Referencia: insertar, actualizar y borrar

```
await personas.insert_one({"nombre": "Ana", "edad": 31})
await personas.insert_many([{"nombre": "Luis"}, {"nombre": "Eva", "edad": 17}])

# update_one: sólo el primer documento que cumple el filtro
await personas.update_one({"nombre": "Ana"}, {"$set": {"edad": 32}})
# upsert=True: si ninguno lo cumple, lo crea
await personas.update_one({"nombre": "Leo"}, {"$set": {"edad": 20}}, upsert=True)

# update_many: todos los que cumplen el filtro
r: UpdateResult = await personas.update_many(
    {"edad": {"$gte": 18}}, {"$set": {"mayor_de_edad": True}}
)
print(r.matched_count, r.modified_count)

await personas.replace_one({"nombre": "Eva"}, {"nombre": "Eva", "edad": 18})
await personas.delete_one({"nombre": "Luis"})
await personas.delete_many({"edad": {"$lt": 18}})
```

Referencia: operadores de actualización

El segundo argumento de `update_one` y `update_many` dice **qué cambiar**; para sustituir el documento entero está `replace_one`

Operador	Efecto	Ejemplo
`$set` / `$unset`	Fija un campo, creándolo si no existe / lo elimina	`{"$set": {"edad": 32}}`
`$inc` / `$mul`	Suma / multiplica	`{"$inc": {"visitas": 1}}`
`$min` / `$max`	Cambia sólo si el nuevo valor es menor / mayor	`{"$max": {"record": 87}}`
`$push`	Añade un elemento a un *array*	`{"$push": {"aficiones": "ajedrez"}}`
`$addToSet`	Lo añade sólo si no está ya	`{"$addToSet": {"aficiones": "ajedrez"}}`
`$pull`	Quita los elementos que cumplen una condición	`{"$pull": {"notas": {"$lt": 5}}}`

Se pueden combinar varios en una misma actualización, que sigue siendo **atómica** para ese documento

Referencia: `aggregate`, etapa a etapa

Un *pipeline* es una lista de etapas: cada una recibe documentos de la anterior y entrega otros a la siguiente

Etapa	Qué hace	En SQL
`$match`	Filtra, con la misma sintaxis que `find`	`WHERE`; tras un `$group`, `HAVING`
`$project`, `$addFields`	Elige, renombra o calcula campos	La lista del `SELECT`
`$group`	Agrupa por `_id` y calcula `$sum`, `$avg`, `$max`…	`GROUP BY` y agregados
`$sort`, `$skip`, `$limit`	Ordena y pagina	`ORDER BY`, `OFFSET`, `LIMIT`
`$lookup`	Trae los documentos relacionados de otra colección	`LEFT JOIN`
`$unwind`	Un documento por cada elemento de un *array*	`JSON_TABLE` (sesión 2)
`$count`	Cuenta los documentos que llegan	`COUNT(*)`
`$out`, `$merge`	Guarda el resultado en una colección	`CREATE TABLE ... AS SELECT`

`aggregate`: la llamada más sencilla

Las tres preguntas mejor puntuadas, con `find` y como *pipeline*:

```
cursor: AsyncCursor = posts.find({"PostTypeId": 1}, {"_id": 0, "Title": 1, "Score": 1})
await cursor.sort("Score", -1).limit(3).to_list()
```

```
pipeline: list[dict[str, Any]] = [
    {"$match": {"PostTypeId": 1}},                  # el filtro de find
    {"$sort": {"Score": -1}},
    {"$limit": 3},
    {"$project": {"_id": 0, "Title": 1, "Score": 1}},    # la proyección de find
]
cursor: AsyncCommandCursor = await posts.aggregate(pipeline)   # sí lleva await
await cursor.to_list()
```

En una etapa, `"$Score"` (con `$`) es **el valor** del campo `Score`; sin `$` sería la cadena `"Score"`

`aggregate`, ejemplo 1: agrupar y contar

```sql
SELECT PostTypeId, COUNT(*) AS n, AVG(Score) AS media
FROM Posts GROUP BY PostTypeId ORDER BY n DESC;
```

```python
cursor: AsyncCommandCursor = await posts.aggregate([
    {"$group": {
        "_id": "$PostTypeId",          # GROUP BY PostTypeId
        "n": {"$sum": 1},              # COUNT(*)
        "media": {"$avg": "$Score"},   # AVG(Score)
    }},
    {"$sort": {"n": -1}},              # ORDER BY n DESC
])
await cursor.to_list()
```

La clave de agrupación siempre se llama `_id`; con `"_id": None` se agrega toda la colección

`aggregate`, ejemplo 2: unir con `$lookup`

Los cinco usuarios con más preguntas, con su nombre:

```
pipeline: list[dict[str, Any]] = [
    {"$match": {"PostTypeId": 1}},                                  # WHERE
    {"$group": {"_id": "$OwnerUserId", "preguntas": {"$sum": 1}}},  # GROUP BY
    {"$sort": {"preguntas": -1}},                                   # ORDER BY
    {"$limit": 5},                                                  # LIMIT
    {"$lookup": {"from": "users", "localField": "_id",              # JOIN Users
                 "foreignField": "Id", "as": "autor"}},
    {"$unwind": "$autor"},                          # el array de un elemento, a subdocumento
    {"$project": {"_id": 0, "nombre": "$autor.DisplayName", "preguntas": 1}},
]
await (await posts.aggregate(pipeline)).to_list()
```

- El orden importa: `$limit` va **antes** de `$lookup`, para unir cinco documentos y no todos
- `$match` al principio puede usar un índice; después de un `$group`, ya no

`aggregate`, ejemplo 3: `HAVING` y `$out`

```sql
CREATE TABLE AutoresFrecuentes AS
SELECT OwnerUserId, COUNT(*) AS preguntas, MAX(Score) AS mejor
FROM Posts WHERE PostTypeId = 1
GROUP BY OwnerUserId HAVING COUNT(*) > 10;
```

```python
await posts.aggregate([
    {"$match": {"PostTypeId": 1}},                    # WHERE: antes de agrupar
    {"$group": {"_id": "$OwnerUserId",
                "preguntas": {"$sum": 1}, "mejor": {"$max": "$Score"}}},
    {"$match": {"preguntas": {"$gt": 10}}},           # HAVING: después de agrupar
    {"$out": "autores_frecuentes"},                   # el resultado, a una colección
])
await db.autores_frecuentes.find_one()
```

Es la misma etapa `$match` las dos veces: lo que cambia es **dónde** está en el *pipeline*

Referencia: índices y planes

```
await users.create_index([("Reputation", 1)])                    # ascendente
await users.create_index([("Reputation", -1), ("Views", -1)]) # compuesto
await posts.create_index([("Id", 1)], unique=True)               # sin duplicados
await users.index_information()                                  # índices que existen
await users.drop_index("Reputation_1")
```

```
plan: dict[str, Any] = await users.find({"Reputation": 100}).explain()
plan["queryPlanner"]["winningPlan"]                    # el plan elegido
plan["executionStats"]["totalKeysExamined"]            # entradas de índice leídas
plan["executionStats"]["totalDocsExamined"]            # documentos leídos
plan["executionStats"]["nReturned"]                    # documentos devueltos
```

- En el plan, `COLLSCAN` es recorrer la colección entera e `IXSCAN`, usar un índice
- Un buen índice acerca `totalDocsExamined` a `nReturned`

Referencias

- M. Kleppmann. *Designing Data-Intensive Applications*. O'Reilly, 2017. Capítulos 2 (modelos de datos), 5 (replicación) y 7 (transacciones)
- D. Pritchett. *BASE: An ACID Alternative*. ACM Queue, 2008
- E. Brewer. *CAP Twelve Years Later: How the "Rules" Have Changed*. IEEE Computer, 2012
- MongoDB. Transacciones, replicación, *change streams* y *pipeline* de agregación
- MongoDB. Equivalencias entre SQL y MongoDB
- PyMongo: API asíncrona